Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory work 4:

Study and Empirical Analysis of Dijkstra and Floyd–Warshall Shortest Path Algorithms

Elaborated:
st. gr. FAF-241 Pleşu Dinu

Verified:
asist. univ. Fiştic Cristofor

Chişinău - 2026

TABLE OF CONTENTS

LIST OF ABBREVIATIONS AND DEFINITIONS

- dense graph - graph with a high number of edges compared with the maximum possible number;
- Dijkstra - single-source shortest-path algorithm for graphs with non-negative weights;
- DP - Dynamic Programming;
- Floyd–Warshall - dynamic-programming algorithm for all-pairs shortest paths;
- KB - kilobytes;
- ms - milliseconds;
- sparse graph - graph with a relatively low number of edges.

INTRODUCTION

Shortest-path algorithms are used to determine minimum-cost paths in weighted graphs. Dijkstra computes the distances from one source vertex to all other reachable vertices, while Floyd–Warshall computes the shortest paths between all pairs of vertices.

These two algorithms are appropriate for empirical comparison because they solve related problems, but rely on different algorithmic strategies. The laboratory therefore studies not only the graph size, but also the effect of sparse and dense structures on the practical performance of the implementations.

1 ALGORITHM ANALYSIS

Objective

Study and analyze Dijkstra and Floyd–Warshall on weighted directed graphs with different sizes and densities.

Tasks

1. study the dynamic-programming method of designing algorithms;
2. implement Dijkstra and Floyd–Warshall in a programming language;
3. perform empirical analysis for sparse and dense graphs;
4. increase the number of nodes and analyze the influence on the algorithms;
5. make a graphical presentation of the obtained data;
6. formulate conclusions.

Theoretical Notes

Empirical analysis is useful when the intention is to observe the practical behavior of algorithms on real inputs and to compare implementations under the same execution environment. In the present laboratory, Dijkstra is used for the single-source shortest-path problem, while Floyd–Warshall is used for the all-pairs shortest-path problem [1, 2].

Dijkstra is expected to scale much better on large graphs, especially when an efficient priority queue is used. Floyd–Warshall is based on a dynamic-programming recurrence and has cubic time complexity, which makes it practical only for substantially smaller graphs [3].

Comparison Metric

The selected metrics were:

– execution time in milliseconds;
– peak memory usage in kilobytes;
– number of reachable distances;
– number of edges and graph density.

Input Format

The benchmark used weighted directed graphs with positive integer weights from 1 to 20. Three graph categories were generated:

- sparse graphs;
- medium-density graphs;
- dense graphs.

For Dijkstra, the tested sizes were 100, 250, 500, 1000, and 1500 nodes. For Floyd–Warshall, the tested sizes were 25, 50, 75, 100, and 125 nodes.

2 IMPLEMENTATION

2.1 Dijkstra

The project contains a classic Dijkstra implementation and an optimized implementation based on a priority queue. Both were benchmarked on the same generated weighted graphs [4].

Algorithm Description

The method follows the idea shown in the next pseudocode.

```
Dijkstra(graph, source):
    set distance[source] = 0
    set all other distances to infinity
    while there are unprocessed vertices:
        choose the vertex with minimum distance
        relax all outgoing edges from that vertex
    return distance array
```

Implementation

The optimized version replaces repeated linear minimum selection with a heap-based priority queue. This modification increases auxiliary memory usage, but reduces running time significantly on large inputs.

Results

After running Dijkstra for each graph size defined in the dataset specifications and recording execution time and memory usage, the representative results are shown in Table 2.1.

Table 2.1 – Dijkstra benchmark at 1500 nodes

Graph type	Algorithm	Time (ms)	Memory (KB)	Edges
dense	Dijkstra_Classic	4562.149	23.66	1461720.0
dense	Dijkstra_Optimized	226.503	209.75	1461720.0
sparse	Dijkstra_Classic	3325.892	23.66	337166.0
sparse	Dijkstra_Optimized	53.918	175.16	337166.0

In Table 2.1 it can be observed that the optimized implementation is substantially faster than the classic one. On the dense graph with 1500 nodes and 1461720 edges, the classic version required 4562.149 ms, while the optimized version required only 226.503 ms. This difference is represented graphically in Figure 2.1.

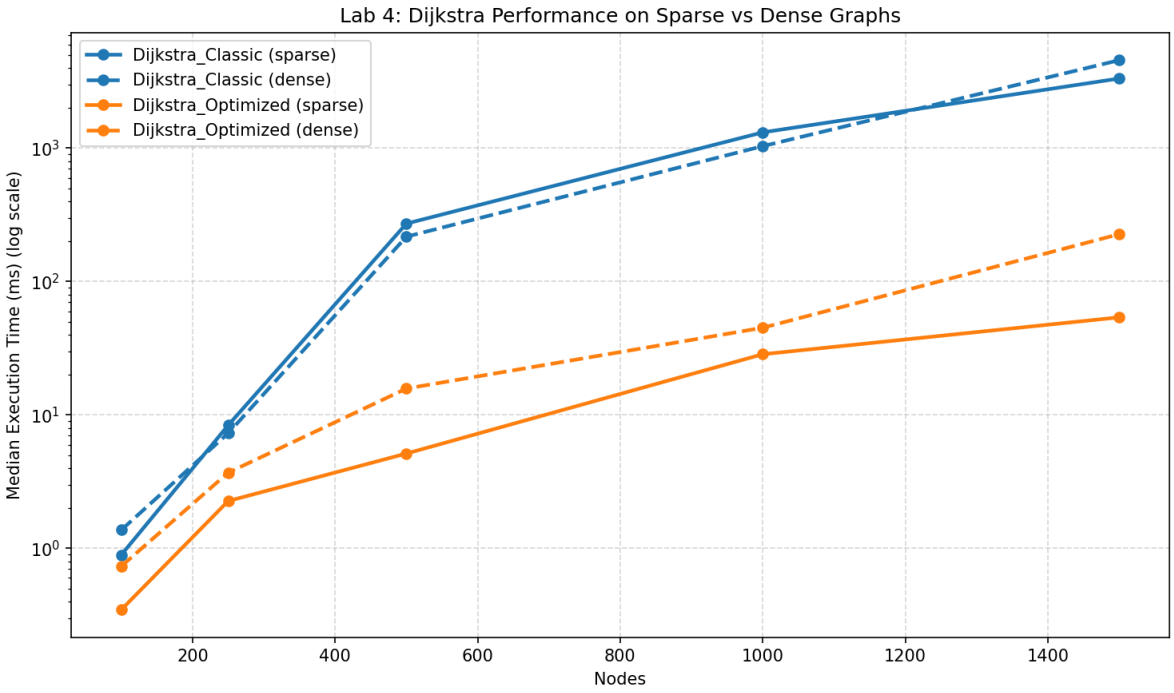

Figure 2.1 – Dijkstra performance on sparse and dense graphs

Figure 2.1 shows clearly that the difference between the two implementations grows as the number of nodes increases. The logarithmic scale is needed because the gap becomes very large on the bigger graphs.

Conclusion

Dijkstra is well suited for large weighted graphs when shortest paths from a single source are required. The priority-queue-based version offers a decisive improvement in performance, especially on larger sparse and dense instances.

2.2 Floyd–Warshall

The project also contains a classic and an optimized Floyd–Warshall implementation. Both solve the all-pairs shortest-path problem and were benchmarked on the same weighted graph categories.

Algorithm Description

The Floyd–Warshall method follows the recurrence shown in the next pseudocode.

```
FloydWarshall(matrix):
    dist = matrix copy
    for k from 1 to n:
        for i from 1 to n:
            for j from 1 to n:
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    return dist
```

Implementation

The optimized version attempts to skip unnecessary updates when intermediate states are not useful. In Python, however, the effect is smaller than for Dijkstra because the algorithm remains cubic and interpreter overhead stays significant.

Results

After executing Floyd–Warshall for each graph size from the second input range, the representative values are shown in Table 2.2.

Table 2.2 – Floyd–Warshall benchmark at 125 nodes

Graph type	Algorithm	Time (ms)	Memory (KB)	Edges
dense	FloydWarshall_Classic	440.023	125.85	10169.0
dense	FloydWarshall_Optimized	728.650	125.80	10169.0
sparse	FloydWarshall_Classic	650.790	125.85	2337.0
sparse	FloydWarshall_Optimized	625.741	125.80	2337.0

In Table 2.2 it can be observed that the difference between the classic and optimized Floyd–Warshall variants is not as dramatic as for Dijkstra. On dense graphs, the classic version remained faster, while on sparse graphs the optimized version produced a small improvement. The corresponding graph is shown in Figure 2.2.

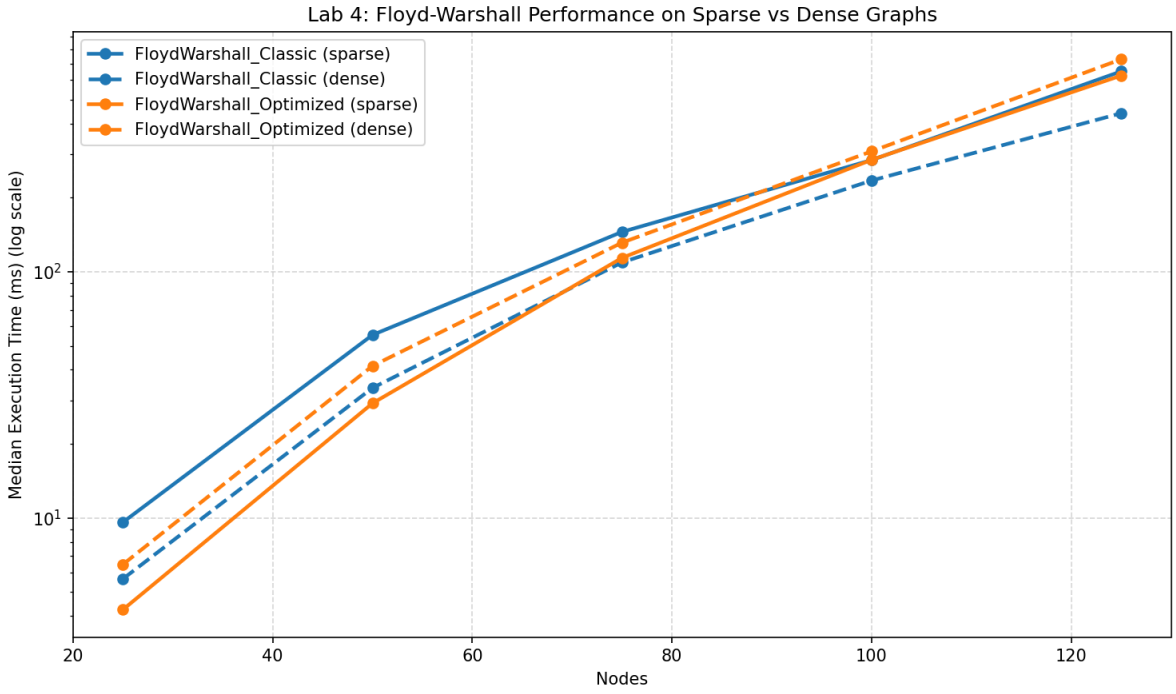

Figure 2.2 – Floyd–Warshall performance on sparse and dense graphs

The graph in Figure 2.2 illustrates the strong growth of running time relative to the number of nodes. Even though the tested node counts are much smaller than for Dijkstra, the increase is already significant because of the cubic complexity of the method.

Conclusion

Floyd–Warshall remains useful when all-pairs shortest paths are required, but the empirical data confirms that it becomes expensive quickly as the number of nodes increases. The optimized implementation offers only moderate gains in Python.

CONCLUSION

The experiments confirm that graph density and graph size strongly influence shortest-path algorithms. Dijkstra is much more scalable for single-source queries, especially in its optimized implementation, while Floyd–Warshall remains suitable only for relatively small all-pairs problems.

The sparse-versus-dense comparison was particularly informative. Dense graphs increased the workload for both methods, but the optimized Dijkstra implementation still maintained a large performance advantage. Overall, the empirical results agree well with the theoretical expectations and provide a clear practical comparison between the two algorithms.

REFERENCES

REFERENCES

[1] GeeksforGeeks. *Dynamic Programming* [online]. [cited 07.05.2026]. Available: geeksforgeeks.org/dynamic-programming.

[2] TutorialsPoint. *Dynamic Programming* [online]. [cited 07.05.2026]. Available: tutorialspoint.com/data_structures_algorithms/dynamic_programming.htm.

[3] Coding Ninjas. *Dijkstra's Algorithm vs Floyd–Warshall's Algorithm* [online]. [cited 07.05.2026]. Available: codingninjas.com/codestudio/library/dijkstras-algorithm-vs-floydwarshalls-algorithm.

[4] GitHub Repository. *Lab 4 source folder* [online]. [cited 07.05.2026]. Available: github.com/Prince-of-Nothing/aa-course-repo/tree/main/Lab4.